

ISAAI

Automatic Report Documentation

Final Report – Summer Semester 2026

Frankfurt University of Applied Sciences – FB2

Module: ISA (Information Systems Architecture)
University: Frankfurt University of Applied Sciences — Faculty 2
Semester: Summer Semester 2026
Project Team: Altay Hennig (1483613)
Christina Malki (1574803)
Sarah Bullinger (1577846)
E-Mail: altay.hennig@stud.fra-uas.de
christina.malki@stud.fra-uas.de
sarah.bullinger@stud.fra-uas.de
Examiner: Mr. Dominik Dietrich
Date: Juli 31, 2026

Contents

1	Introduction	3
1.1	Project Context	3
1.2	Dual Module Approach	3
1.3	Project Goal	3
1.4	Project Organization	4
2	Management Summary	4
2.1	Solution Overview	4
2.2	Key Result	4
3	Problem Description	4
3.1	As-Is State (CURRENT State)	4
3.2	Identified Weaknesses	5
3.3	Risk and Control Gap	5
3.4	Report Types	5
4	Solution Design	5
4.1	Architecture Evolution: CURRENT → GOAL	5
4.2	GOAL (Realized State)	6
4.3	Power Automate Flow — Technical Architecture	7
4.4	Technology Stack	7
5	Challenges	7
5.1	Power Automate	7
5.2	DMS Integration (Out of Scope)	7
6	Lessons Learned and Findings	8
6.1	Architecture and Modeling	8
6.2	Technology and Automation	8
7	Outcome and Conclusion	8
7.1	Project Result	8
7.2	Quantitative Benefits	8
7.3	Conclusion	9
8	Magic Charts	9
8.1	Concept	9
8.2	Implemented Scripts	10
9	Sources	10
9.1	Architecture and Modeling	10
9.2	Compliance and Governance	10
9.3	Development Tools	10
9.4	Project Resources	10

1. Introduction

The ISAAI project — short for *Information Systems Architecture and Integration* — was carried out during the summer semester of 2026 at the Frankfurt University of Applied Sciences. It spans two complementary modules: *Information Systems Architecture (ISA)* and *Architecture and Integration (A+I)*, both supervised by Prof. Dr. Dominik Dietrich in Faculty 2. The central motivation behind the project is the automation of a daily, compliance-critical business process that, in its current form, consumes significant operational effort and is prone to human error.

1.1 Project Context

In the day-to-day operations of a financial service provider, two types of structured reports arrive every morning via e-mail. **Report Type A** captures the previous business day's trade-level details in a daily snapshot, while **Report Type B** provides a month-to-date cumulative summary of trader activity. Both reports are delivered as XML files packaged inside ZIP archives and contain compliance-relevant markers — so-called *DayViol* and *MonthViol* flags — that indicate whether trading thresholds have been breached.

Until now, this entire workflow has been performed manually: an employee opens the incoming e-mail, extracts the ZIP archive, imports the XML data into a spreadsheet, visually inspects the violation markers, creates evidence documents, archives them, and escalates any issues to a supervisor. The whole cycle takes between 30 and 40 minutes each day — time that could be spent on higher-value analytical tasks instead.

1.2 Dual Module Approach

Because the project serves two university modules simultaneously, the work is organized along two tracks that reinforce each other. The A+I track focuses on the operative process automation itself: building the end-to-end pipeline that ingests Report A and Report B, evaluates compliance rules, generates evidence files, and manages exception governance. The ISA track, on the other hand, contributes the enterprise architecture perspective — ArchiMate modeling across three transformation stages, BPMN process models, and a parallel university deliverable covering invoice analysis with chart automation.

Table 1: Module Comparison

Module	Focus	Technology
A+I	Enterprise Architecture, ArchiMate Modeling, Invoice and Chart Automation	ArchiMate, Power Automate, PowerPoint
ISA	Operative Process Automation, XML Validation, Exception Governance, GOAL BPMN	Power Automate, PowerPoint, Camunda Modeler

This dual-track structure ensures that the project is both architecturally grounded and practically relevant. The ArchiMate diagrams provide the strategic “big picture,” while the Power Automate implementation delivers a working, deployable solution.

1.3 Project Goal

The overarching goal of the project can be stated concisely: *eliminate the 30–40 minutes of daily manual effort while preserving full auditability and human oversight for exceptions*. More precisely, the system should automatically detect incoming report e-mails, extract and parse the XML content, evaluate the compliance rules in parallel for both report types, generate professionally formatted evidence documents, and store everything in a traceable audit trail on SharePoint. Human intervention should be required only when an actual rule violation is detected — the classic *human-in-the-loop* pattern.

To ensure this goal is concrete and measurable, it was formulated according to the SMART framework:

Table 2: SMART Goal Framework

Specific	Measurable	Achievable	Relevant	Time-bound
Report A/B XML Pipeline + parallel ISA Invoice Pipeline	End-to-End Automation for the standard path; Exceptions measurable	GitHub Mock existing BPMN/ArchiMate models	CI/CD, APIs, White-box inter-faces; Audit Trail; 30–40 min/day time savings	Deadline June 30, 2026

1.4 Project Organization

The project team consists of three core members, each with clearly defined responsibilities:

Table 3: Project Organization

Role	Person / Unit
Sponsor and Academic Lead	Dominik Dietrich — FRA UAS / FB2
Architecture and Software Engineering	Christina Malki, Altay Hennig — Modules ISA and A+I
Operative Owner	Sarah Bullinger — Project Management Operations

2. Management Summary

2.1 Solution Overview

The ISAAI system replaces the entire manual report processing workflow with a fully automated **Microsoft Power Automate Cloud Flow**. When an e-mail with the subject “Financial Report” arrives in the designated mailbox, the flow is triggered automatically. It extracts the ZIP attachment, unpacks both XML report files, and processes them in parallel through dedicated parsing and validation branches.

For each report type, the system applies XPath-based rule evaluation to detect violation markers. If both reports pass without violations, the flow generates professionally styled XLSX evidence documents, saves them to a SharePoint Document Library, drafts a summary e-mail for the relevant stakeholders, sends it, and archives a copy of the sent message. If a rule violation is detected, the flow follows the exception governance path: it generates an exception alert that is escalated to the Operations Supervisor, who then reviews the violation and executes any necessary governance actions.

Throughout this process, every step is logged in a SharePoint List that serves as the operational database. This creates a seamless audit trail satisfying both internal compliance requirements and the traceability demands of external audits.

2.2 Key Result

The automation eliminates the entire daily manual effort of 30 to 40 minutes while ensuring full auditability. Human control is preserved where it matters most — for exception handling and governance sign-off — but the routine, error-prone manual steps are completely removed from the process.

3. Problem Description

3.1 As-Is State (CURRENT State)

In the current state, daily financial reports are processed entirely by hand. The workflow begins when an employee receives a daily e-mail containing a ZIP archive with two XML report files. The employee manually unpacks the archive, opens the XML files, and imports the data into a spreadsheet application. There, violation markers and threshold values are checked visually — a task that requires concentration and domain knowledge, yet is highly repetitive and offers little room for value-adding analysis.

Once the check is complete, the employee creates evidence documents to record the results, formats them according to internal standards, and files them in the company's document management system. If any violations are found, they must be flagged and escalated to a supervisor, who reviews the case and decides on appropriate governance actions. Finally, a summary is distributed to the relevant stakeholders. This entire cycle — extraction, validation, documentation, archiving, and distribution — takes approximately 30 to 40 minutes every business day.

The corresponding BPMN process model for the **CURRENT** state is available in the project repository as a .bpmn file, documenting the manual flow in detail.

3.2 Identified Weaknesses

Several structural weaknesses make this manual process particularly problematic. The most obvious is the **time consumption**: dedicating 30 to 40 minutes of skilled labor every day to a routine task is an inefficient use of resources. Beyond time, the process involves a significant **media break** — employees must switch between the e-mail client, a file explorer, a spreadsheet application, and the document management system, each transition creating opportunities for errors and information loss.

There is also a **lack of system enforcement**: no automated mechanism ensures that the upload-to-sign-off workflow is followed correctly every time. This creates compliance risk and makes it difficult to trace exactly what happened during any given processing run. Finally, the reliance on **human judgment** for routine checks introduces the risk of inconsistent outputs, forgotten steps, and quality defects that could surface during audits.

Table 4: Identified Weaknesses in As-Is Process

Weakness	Description	Impact
Time Consuming	30–40 minutes of daily manual effort	Loss of productivity, resource binding
Media Break	Switching between Collaboration Storage and Document Repository	Susceptibility to errors, loss of information
Lack of System Enforcement	No system-supported upload-to-sign-off process	Compliance risk, lack of traceability
Human Errors	Incorrect fields, inconsistent outputs, forgotten steps	Quality defects, audit risks

3.3 Risk and Control Gap

Manual processing multiplies traceability gaps and increases the risk of unauthorized changes or overlooked violations. Without automation, there is no standardized audit trail — each processing run depends on the individual employee's diligence and familiarity with the procedure. This lack of standardization exacerbates compliance risk and makes it difficult to demonstrate regulatory conformity during audits.

3.4 Report Types

The two report types differ in their temporal scope and analytical purpose, but share the same delivery mechanism and compliance-relevant structure:

4. Solution Design

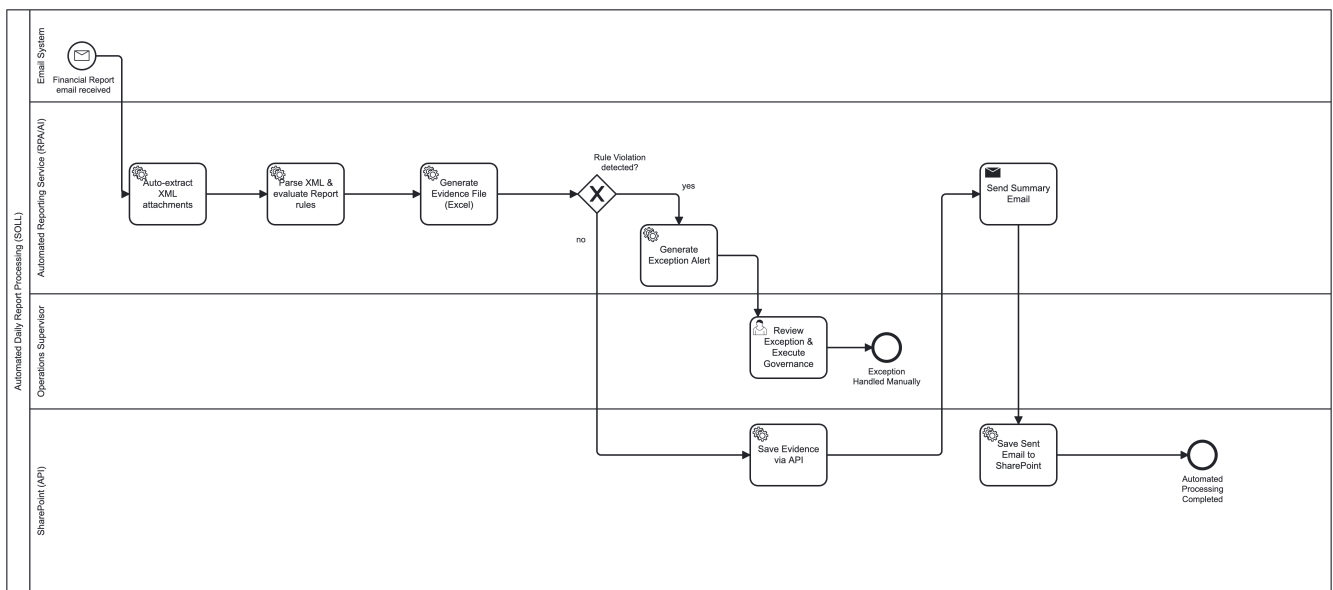
4.1 Architecture Evolution: **CURRENT** → **GOAL**

The project follows a three-stage architecture evolution model that is common in enterprise architecture practice. The **CURRENT** state documents the manual as-is process. The **VISION** state represents the ideal fully automated target, including integration with the company's internal Document Management

Table 5: Report Types A and B Characteristics

Attribute	Report Type A (Daily)	Report Type B (Monthly)
Coverage Period	Previous business day (Trade-Level Detail)	Month-to-Date cumulative trader summary
Delivery Frequency	Daily, at the agreed processing time	Daily, together with Report Type A
Violation Marker	DayViol	MonthViol
Purpose	Detection of same-day trade exceptions	Tracking monthly cumulative trader breach trends

System via UI scraping or RPA. The **GOAL** state is the pragmatically scoped, actually implemented solution — it delivers full automation of the core workflow while deliberately excluding the DMS integration, which proved impractical within the project constraints.

Figure 1: Strategic BPMN GOAL Process [[Download .bpmn](#)]

This three-tier approach is documented in both ArchiMate 3.1 models (available as `.archimate` files) and BPMN 2.0 process models (available as `.bpmn` files) in the project repository. The models trace the transformation path from manual operations to automated processing, making every architectural decision transparent and traceable.

The project utilizes already realized automation capabilities provided by Microsoft as part of their Office 365 suite. Rather than building custom infrastructure from scratch, the solution leverages Power Automate's native connectors for Exchange, SharePoint, and XML processing. This approach minimizes development effort while maximizing reliability and maintainability.

4.2 GOAL (Realized State)

The GOAL state represents what was actually built and deployed. It is a fully functional Microsoft Power Automate Cloud Flow that handles the complete report processing pipeline — from e-mail detection through XML parsing, rule evaluation, evidence generation, and SharePoint archiving to stakeholder notification.

The DMS integration that was envisioned in the VISION state was deliberately excluded from the GOAL scope. SharePoint serves as the sole Evidence Repository, which is both simpler and more maintainable than the originally planned DMS integration via UI scraping. This pragmatic scope reduction is a conscious architectural decision, not a compromise in functionality: SharePoint fully satisfies the requirements for traceable evidence storage and audit trail documentation.

4.3 Power Automate Flow — Technical Architecture

The flow *ISAAI – Daily Report Processing* is structured into six sequential phases and is available as an importable Power Automate package in the repository under `src/flow/ISAAI-DailyReport-Processing_20260625165442.zip`. The six phases cover trigger detection, ZIP extraction, parallel XML parsing, rule-based validation, evidence generation with SharePoint archiving, and summary e-mail distribution. Each phase is designed to be independently testable and produces traceable outputs that are logged in the SharePoint List.

4.4 Technology Stack

The solution is built on a carefully chosen set of technologies that balance capability with accessibility:

Table 6: Technology Stack

Component	Technology
Automation	Microsoft Power Automate (Cloud Flow)
Data Storage	SharePoint Online (Lists + Document Library)
Evidence Format	XLSX (Excel)
Architecture Modeling	ArchiMate 3.1 (Archi)
Process Modeling	BPMN 2.0 (Camunda Modeler)
Local Simulation	Python 3, openpyxl, python-pptx
Version Control	Git / GitHub
Artificial Intelligence	Gemini, Claude, Antigravity, CursorAI

The choice of Power Automate as the automation platform was driven by the organization's existing Microsoft 365 licensing. Using a low-code platform rather than custom code significantly reduces the maintenance burden and makes the solution accessible to non-developers in the operations team. Python was used for local simulation and testing, generating reproducible mock data for 30 test runs to validate the logic before deployment.

5. Challenges

5.1 Power Automate

Working with Microsoft Power Automate presented several practical challenges that are worth documenting for future projects. The platform's AI-assisted flow generation (Microsoft Copilot) was unable to generate or import complex branching flows reliably. Specifically, the parallel processing of two report types with conditional error handling and exception governance exceeded the complexity that Copilot could handle in a single generation step. As a result, the entire flow had to be designed and built manually in the Power Automate Designer, step by step.

A detailed guide documenting the manual build process was created and stored in the repository under `docs/guides/SHAREPOINT_FLOW_GUIDE.md`. This guide serves both as internal documentation and as a reference for anyone who needs to modify or extend the flow in the future.

5.2 DMS Integration (Out of Scope)

The original VISION called for full integration with the company's internal Document Management System, including UI scraping and RPA-based automation to upload evidence documents and obtain sign-off approvals. During the analysis phase, however, several factors made this integration impractical for the project scope.

First, UI scraping is inherently fragile: any change to the DMS user interface would break the automation, requiring constant maintenance. Second, no stable API to the DMS was available, which meant that all

interaction would have had to go through the graphical user interface — a fundamentally unreliable approach for production use. Third, and most importantly, SharePoint already provides all the functionality needed for evidence storage and audit trail documentation.

The decision to exclude DMS integration was therefore not a failure to meet requirements, but a deliberate, well-reasoned architectural decision that prioritizes reliability and maintainability over theoretical completeness. This kind of pragmatic scope management is a valuable skill in both academic and corporate settings.

6. Lessons Learned and Findings

6.1 Architecture and Modeling

One of the most significant insights from this project concerns the role of ArchiMate modeling in practice. While the three-tier architecture evolution (CURRENT → VISION → GOAL) was conceived primarily as a university deliverable, the models proved to be unexpectedly valuable as a communication tool.

When presenting the project to stakeholders, the ArchiMate diagrams provided a clear, visual way to explain the transformation path from manual to automated processing. Each layer of the model — from the business motivation through the application architecture to the technology stack — told a coherent story about why the project exists, what it changes, and how it delivers value. This experience suggests that in professional practice, architecture models are most effective when they are used as communication instruments rather than purely technical documentation.

The project also revealed an interesting distinction between ArchiMate usage for internal strategy discussions versus delivery-oriented communication. The same model can be “sliced” differently depending on the audience: a governance-focused view emphasizes compliance controls and exception paths, while a technology-focused view highlights integration points and data flows. This compartmentalization approach can significantly increase the efficiency and clarity of stakeholder communications.

6.2 Technology and Automation

Power Automate proved to be well-suited for the defined use case. The combination of an e-mail trigger, ZIP extraction, XML parsing, rule-based validation, and SharePoint storage fits naturally into Power Automate’s connector-based architecture. Each of these operations corresponds to a native connector or a well-documented action, which minimized the amount of custom logic needed.

However, the project also demonstrated the limitations of low-code platforms. Complex branching logic — particularly parallel processing with error handling on multiple branches — pushes Power Automate to the edge of its design paradigm. The visual flow designer becomes unwieldy at a certain level of complexity, and debugging parallel branches requires careful attention to variable scoping and error propagation. For future projects with even more complex logic, a hybrid approach combining Power Automate for orchestration with Azure Functions for complex processing steps might be worth considering.

7. Outcome and Conclusion

7.1 Project Result

The ISAAI project achieved all of its defined goals within the planned timeline. The following table provides a concise overview of each goal and its completion status:

7.2 Quantitative Benefits

Beyond the qualitative improvements in process reliability and auditability, the project also delivers measurable economic benefits. A cost-benefit analysis based on the Function Point and COCOMO estimation methods puts the total implementation effort at approximately 323 working hours. Including

Table 7: Project Results Overview

Goal	Status	Description
Full automation of the standard path	Achieved	E-Mail, ZIP, XML, Validation, Evidence, SharePoint, Notification
Exception Governance	Achieved	Human-in-the-loop for violations with supervisor escalation
Audit Trail	Achieved	Seamless documentation in SharePoint List + Document Library
ArchiMate Modeling (3 stages)	Achieved	CURRENT, VISION, GOAL each as ArchiMate models (.archimate)
BPMN Process Modeling (3 stages)	Achieved	CURRENT, VISION, GOAL each as BPMN diagrams (.bpmn)
Power Automate Flow	Achieved	Importable flow package in the repository
Local Simulation	Achieved	30 reproducible test runs with Python scripts
ISA Module (parallel)	Achieved	Invoice receipt, JSON/CSV, Chart Automation
Presentation Material	Achieved	Consolidated Findings Deck + Module Presentations

allowances for project coordination, change management, training, and a reasonable contingency buffer, the estimated total cost comes to **9,656.40 €**.

This investment pays for itself through the daily elimination of 30 to 40 minutes of manual labor. Over the course of a year, this amounts to roughly 130 to 170 hours of freed-up capacity — operational time that can be redirected toward higher-value activities such as analytical review, process improvement, and strategic planning.

7.3 Conclusion

The ISAAI project demonstrates convincingly how the combination of enterprise architecture methodology (ArchiMate, BPMN) and low-code automation (Power Automate, SharePoint) can completely transform an operational process. The systematic documentation of the transformation path across three architecture stages — CURRENT, VISION, and GOAL — ensures that every decision is traceable and that the solution remains comprehensible and maintainable over time.

The deliberate scope reduction from VISION to GOAL, specifically the exclusion of DMS integration, also illustrates an important practical lesson: pragmatic architecture decisions are just as important in a university project as they are in corporate practice. It is better to deliver a complete, reliable solution within a well-defined scope than to attempt a more ambitious target that cannot be fully realized. The SharePoint-based solution meets all functional requirements while providing a solid, extensible foundation for future enhancements — including, potentially, the DMS integration that was deferred from the original VISION.

8. Magic Charts

8.1 Concept

A distinctive feature of the ISAAI project is the *Magic Charts* module — a collection of Python scripts that automate the creation of visualizations and presentations from the processed report data. The underlying idea is straightforward: since the evidence files produced by the Power Automate Flow are stored in structured, machine-readable formats (XLSX), they can serve as input for automated chart generation and presentation building. This transforms raw compliance data into professional, meaningful visual artifacts without any manual formatting effort.

8.2 Implemented Scripts

The Magic Charts module comprises five interconnected scripts, each serving a specific purpose in the data-to-presentation pipeline:

Test Data Generation (`simulate_runs.py`) produces reproducible mock XML and ZIP test data for 30 simulated business days. This script was essential during development, as it allowed the team to test the entire pipeline end-to-end without depending on live report data.

Local Flow Processing (`local_flow_processor.py`) simulates the full Power Automate Flow offline and generates professionally styled evidence files. This local simulation capability was invaluable for development and debugging, enabling rapid iteration without deploying to the cloud environment.

Consolidated Findings Presentation (`generate_findings_presentation.py`) aggregates the results from all test runs into a single, coherent findings presentation that provides a high-level overview of compliance trends and exception patterns.

Boardroom Deck (`build_boardroom_deck.py`) generates a management-ready presentation with executive-level visualizations and key metrics, suitable for governance meetings and stakeholder briefings.

Demo Flow (`demo_flow.py`) provides an interactive demonstration mode that walks through a single processing run with visual output, making it easy to showcase the system's capabilities to non-technical audiences.

9. Sources

9.1 Architecture and Modeling

1. ArchiMate 3.1 Specification. The Open Group. <https://pubs.opengroup.org/architecture/archimate3-doc/>
2. Archi — ArchiMate Modelling Tool. <https://www.archimatetool.com/>
3. BPMN 2.0 Specification. Object Management Group (OMG). <https://www.omg.org/spec/BPMN/2.0/>
4. Camunda BPMN Modeler. Camunda Services GmbH. <https://camunda.com/>

9.2 Compliance and Governance

1. EU AI Act — Regulation (EU) 2024/1689. European Parliament and Council. <https://eur-lex.europa.eu/eli/reg/2024/1689/oj>
2. General Data Protection Regulation (GDPR) — Regulation (EU) 2016/679. European Parliament and Council. <https://eur-lex.europa.eu/eli/reg/2016/679/oj>

9.3 Development Tools

1. Python 3 Documentation. Python Software Foundation. <https://docs.python.org/3/>
2. openpyxl — A Python library to read/write Excel 2010 xlsx/xlsm/xltx/xltn files. <https://openpyxl.readthedocs.io/>
3. python-pptx — Python library for creating PowerPoint files. <https://python-pptx.readthedocs.io/>
4. GitHub — Collaborative Development Platform. <https://github.com/>

9.4 Project Resources

1. ISAAI GitHub Repository. All attachments can be accessed via the following repository: <https://github.com/pollo60/-ISAAI-Automatic-Report-Documentation>
2. SharePoint Site — ISAAI Daily Report Processing. <https://studfrauasde.sharepoint.com/sites/ISAAIDailyReportProcessing>

Project ownership: Prof. Dr. Dominik Dietrich (sponsor) | Christina Malki, Altay Hennig (engineering) | Sarah Bullinger (operations)